

# Hadoop MapReduce

Interview Prep Guide — Concepts, Phases & Architecture



Big Data Engineering Reference — How Distributed Processing Actually Works in Hadoop

# 🔥 What is MapReduce?

**MapReduce** is Hadoop's programming model for processing huge datasets in parallel across a cluster. It breaks a job into two simple steps — **Map** (transform data into key-value pairs) and **Reduce** (aggregate those pairs into a result) — and lets the framework handle parallelization, fault tolerance, and data movement for you.

## The Core Idea

Instead of writing complex distributed-systems code yourself, you write two functions — a **Mapper** and a **Reducer** — and Hadoop automatically splits your input, runs many Mappers in parallel across the cluster (ideally on the nodes where the data already lives), shuffles the results, and runs Reducers to produce the final output.

## Why It Matters (Even With Spark Around)

| Reason                     | Why it's still asked about in interviews                                                    |
|----------------------------|---------------------------------------------------------------------------------------------|
| Foundational model         | Spark, Hive, and Pig all build on the same Map → Shuffle → Reduce mental model              |
| Tests distributed thinking | Interviewers use it to check you understand parallelism, data locality, and fault tolerance |
| Still used in production   | Many legacy pipelines and some batch ETL jobs still run on native MapReduce                 |
| Underlies YARN             | Understanding MapReduce helps you understand how YARN schedules any distributed job         |

💡 **Interview framing:** If asked "why not just use Spark," a strong answer acknowledges Spark's in-memory speed advantage but shows you understand MapReduce's disk-based, fault-tolerant model underneath it all — that's usually what's really being tested.

# ⚡ The MapReduce Phases

Every MapReduce job flows through five stages: **Input Split** → **Map** → **Shuffle & Sort** → **Reduce** → **Output**. Understanding what happens at each stage — and which parts run in parallel — is the most commonly tested concept.

**INPUT** The input file is split into fixed-size **InputSplits** (usually aligned to HDFS blocks). Each split is fed to one Mapper.

**MAP** Each Mapper reads its split as records, applies your map function, and emits intermediate **key-value pairs**. Mappers run in parallel, ideally on the same node that already holds the data (data locality).

**SHUFFLE** Intermediate pairs are **partitioned** by key (so all values for a key go to the same Reducer), sorted, and transferred across the network from Mapper nodes to Reducer nodes. This is usually the most expensive phase.

**REDUCE** Each Reducer receives all values for its assigned keys (already sorted), applies your reduce function, and emits the final output.

**OUTPUT** Reducer output is written to HDFS, typically one output file per Reducer (part-r-00000, etc.).

## Two Optional but Important Extras

| Component   | Role                                                                                                                                                                         |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Combiner    | A "mini-reducer" that runs locally on the Mapper's output before shuffling, to reduce the amount of data sent across the network (e.g. pre-summing counts)                   |
| Partitioner | Decides which Reducer each key goes to. Default is a hash partitioner ( <code>hash(key) % numReducers</code> ); custom partitioners control load balancing or grouping logic |

⚠ **Classic interview question:** "Is the Combiner guaranteed to run?" — No. Hadoop may run it zero, one, or multiple times per Mapper, so it must be logically equivalent whether applied or skipped (e.g. sum and count work; average alone does not, without care).

# 📄 Classic Example — Word Count

The canonical MapReduce example: count how many times each word appears across a huge set of text files.

## Map Function (pseudocode)

```
// input: (line_number, line_of_text)
function map(key, value):
  for word in value.split():
    emit(word, 1)
```

## Reduce Function (pseudocode)

```
// input: (word, [1, 1, 1, ...])
function reduce(word, counts):
  total = sum(counts)
  emit(word, total)
```

## Walkthrough

Input text: "the cat sat" / "the dog ran"

| Stage                         | Output                                        |
|-------------------------------|-----------------------------------------------|
| Map (per line)                | (the,1)(cat,1)(sat,1)   (the,1)(dog,1)(ran,1) |
| Shuffle & Sort (group by key) | the:[1,1] cat:[1] sat:[1] dog:[1] ran:[1]     |
| Reduce (sum per key)          | the:2, cat:1, sat:1, dog:1, ran:1             |

💡 **Interview tip:** Be ready to draw or narrate this exact word-count flow from memory — it's the single most common whiteboard question for MapReduce.

# ⚙️ Execution Architecture (on YARN)

Modern Hadoop (2.x/3.x) runs MapReduce jobs on top of **YARN**, which handles resource allocation and scheduling — this replaced the old Hadoop 1.x JobTracker/TaskTracker model.

## Key Players

- ResourceManager** — cluster-wide master. Tracks available resources and schedules applications (runs on the master node).
- ApplicationMaster** — one per job. Negotiates resources from the ResourceManager and coordinates the job's Map and Reduce tasks.
- NodeManager** — one per worker node. Manages resources and monitors containers on that node, reports back to the ResourceManager.
- Container** — the actual execution unit (CPU + memory allocation) where a Map or Reduce task runs.

## Job Lifecycle (Simplified)

| Step | What happens                                                                                                    |
|------|-----------------------------------------------------------------------------------------------------------------|
| 1    | Client submits the job to the ResourceManager                                                                   |
| 2    | ResourceManager allocates a container to launch the job's ApplicationMaster                                     |
| 3    | ApplicationMaster requests containers for Map and Reduce tasks                                                  |
| 4    | NodeManagers launch the requested containers and run the tasks                                                  |
| 5    | Tasks report progress back to the ApplicationMaster; on completion, the AM reports final status and deregisters |

## Fault Tolerance

If a Map or Reduce task fails, it's simply **re-executed** on another node (up to a configurable number of attempts). If a whole node fails, the ResourceManager reschedules that node's work elsewhere. Because Map task output is written to local disk (not sent immediately), failed Reducers can safely re-pull data without re-running Mappers, in most cases.

⚠️ **Classic interview question:** "What happens if a task fails multiple times?" — After a configurable number of retries (default 4), the whole job is marked as failed. Speculative execution can also run backup copies of slow ("straggler") tasks in parallel and use whichever finishes first.

# Performance & Tuning Concepts

Interviewers often probe whether you understand *why* MapReduce jobs are slow and how to speed them up — these are the concepts to know cold.

| Concept                      | Why it matters                                                                                                                        |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data locality</b>         | Running the Map task on the node that already holds the data block avoids network transfer — the scheduler tries hard to achieve this |
| <b>Number of Reducers</b>    | Too few → bottleneck; too many → excessive small output files. Set via <code>job.setNumReduceTasks()</code>                           |
| <b>Combiner usage</b>        | Reduces network I/O in the shuffle phase by pre-aggregating on the Map side                                                           |
| <b>Compression</b>           | Compressing intermediate Map output (e.g. Snappy, Gzip) cuts shuffle-phase network and disk I/O                                       |
| <b>Speculative execution</b> | Runs duplicate copies of slow tasks on other nodes to avoid one straggler delaying the whole job                                      |
| <b>Input split size</b>      | Too many small splits create excessive task overhead; too few large splits reduce parallelism                                         |

💡 **Interview tip:** The shuffle phase is almost always the performance bottleneck people ask about — know that Combiners, compression, and a sensible Reducer count are the standard levers to pull.

# Rapid-Fire Interview Q&A

## Q: What are the two main functions a developer writes in MapReduce?

The Map function (transforms input records into key-value pairs) and the Reduce function (aggregates values grouped by key).

## Q: What is the shuffle phase and why is it expensive?

It's the process of partitioning, sorting, and transferring Mapper output across the network to the correct Reducer — expensive because it involves disk I/O and network transfer between nodes.

## Q: What's the difference between a Combiner and a Reducer?

A Combiner runs locally on the Mapper's output to reduce data before shuffling; a Reducer runs after shuffling on the globally grouped data. A Combiner is often literally the same code as the Reducer, but isn't guaranteed to run.

## Q: What determines how many Mappers run for a job?

Generally the number of InputSplits, which is usually close to the number of HDFS blocks in the input data.

## Q: How do you control the number of Reducers?

Explicitly, via `job.setNumReduceTasks(n)` — unlike Mapper count, it isn't automatically derived from input size.

## Q: What is a Partitioner?

The component that decides which Reducer each key-value pair goes to, ensuring all values for the same key land on the same Reducer. Default is a hash partitioner.

## Q: What replaced JobTracker/TaskTracker in modern Hadoop?

YARN's ResourceManager and NodeManager, with a per-job ApplicationMaster replacing the centralized JobTracker's scheduling responsibilities.

## Q: What is speculative execution?

Running duplicate copies of a slow-running ("straggler") task on different nodes in parallel, then using whichever copy finishes first and killing the rest.

## Q: Why is data locality important?

Moving computation to the data (rather than data to computation) avoids expensive network transfer, which is the key efficiency principle behind Hadoop's design.

## Q: When would you choose MapReduce over Spark, or vice versa?

Spark is generally preferred for iterative or interactive workloads due to in-memory processing; MapReduce (or Spark in disk-heavy mode) can still suit simple, very large one-pass batch jobs where memory is a constraint, or in legacy environments already built around it.

## Quick Recap

- **MapReduce** = a two-phase (Map, Reduce) programming model for parallel, fault-tolerant batch processing across a cluster
- **Flow:** Input Split → Map → Shuffle & Sort → Reduce → Output, with optional Combiner and Partitioner steps
- **Runs on YARN:** ResourceManager + ApplicationMaster + NodeManager + Containers
- **Fault tolerance:** failed tasks are simply re-run elsewhere; speculative execution handles stragglers
- Know the word-count example cold, and be ready to explain the shuffle phase and Combiner behavior in detail